

---

# Trade-offs in Character N-gram Tokenization: Attention vs Embedding

---

Adacus Greene  
DNU Lab, Dickinson College

## Abstract

Tokenization scheme design is a critical yet often overlooked component of model development, as suboptimal choices can dramatically increase training time and computational cost. This work empirically investigates the trade-offs inherent in character N-gram tokenization by comparing four schemes (1-Char, 2-Char, 3-Char, and 4-Char) on a controlled reversal task with a 6-character alphabet. We train decoder-only transformers on 10,000 structured reversal equations and measure total computational cost to convergence. Our results reveal a U-shaped relationship between tokenization granularity and training efficiency: 1-Char tokenization requires approximately 647 TFLOPs due to quadratic attention scaling over long sequences, while 2-Char tokenization achieves the optimal cost of 44 TFLOPs—a  $14.67\times$  reduction. Coarser schemes (3-Char and 4-Char) incur increased costs of 55 and 95 TFLOPs respectively, driven by exponential growth in embedding parameters. These findings demonstrate that tokenization choice can be a dominant factor in training efficiency and should receive careful consideration during model design.

## 1 Introduction

Tokenization is a foundational design choice in neural language modeling. Practitioners often default to simple schemes without rigorous analysis of their data’s structure: character-level tokenization (1-Char), where each unique character becomes a token, or Byte-Pair Encoding (BPE), which iteratively merges the most frequent adjacent token pairs to build a vocabulary of size  $k$ . While 1-Char tokenization is simple to implement and requires no vocabulary engineering, it carries a substantial computational penalty. In our experiments on a character reversal task, 1-Char tokenization required 647 TFLOPs to converge to  $> 99\%$  validation accuracy, compared to 44 TFLOPs for 2-Char tokenization; a  $14.67\times$  increase in training cost for choosing the seemingly natural option.

This dramatic difference motivates a fundamental question: *What drives the efficiency gap between tokenization schemes, and how can we identify the optimal choice?* We hypothesize that two competing forces govern this trade-off. The attention mechanism scales quadratically with sequence length, creating a penalty as sequences grow longer. Conversely, the embedding layer scales exponentially with vocabulary size, creating a penalty as token granularity increases. On a controlled task with a 6-character alphabet, we demonstrate this trade-off empirically.

### Contributions:

1. Empirically identify 2-Char tokenization as optimal for a 6-character alphabet with input length 144, requiring 44.16 TFLOPs with the lowest variance ( $\sigma = 9.87$ ).
2. Quantify the efficiency trade-offs: explain the  $14.67\times$  speedup from 1-Char to 2-Char (attention bottleneck) and the  $1.8\times$  slowdown from 3-Char to 4-Char (embedding explosion).

This work demonstrates that tokenization choice is not merely an implementation detail but a critical lever for computational efficiency—one that deserves systematic analysis during model design.

## 2 Related Work

This report describes the output of an independent research project in the DNU Lab of Dickinson College. By design, DNU Lab projects focus on conducting research rather than surveying existing work. In this section, we describe two pieces of related work. This serves as an initial step toward a more complete literature survey.

**Subword Tokenization for Natural Language:** Sennrich et al. (2016) introduced Byte-Pair Encoding (BPE) to neural machine translation, establishing subword tokenization as the standard approach for handling open vocabularies and non-uniform natural language distributions. While highly effective for natural languages, BPE relies on frequency-based merge heuristics that may not align with domain-specific, non-natural data. In contrast, our work focuses on highly structured algorithmic data—specifically, character reversal tasks—where fixed character  $N$ -gram schemes allow us to isolate and systematically analyze the core trade-offs of token granularity without frequency-driven vocabulary bias.

**Tokenization in Non-Natural Sequence Modeling:** Dotan et al. (2024) investigated the impact of character-level, BPE, and domain-specific tokenization schemes on transformer performance for biological sequences. Similar to our observations, they identified a fundamental trade-off between sequence length and vocabulary size that substantially impacts training duration. Our work extends beyond these empirical findings in two key ways: by utilizing a tightly controlled 6-character alphabet, we can precisely quantify vocabulary growth ( $6^c$ ), and we isolate the precise computational drivers of this balance—specifically, the quadratic attention overhead against exponential embedding parameter scaling—explaining the U-shaped efficiency curve observed in Figure 1.

## 3 Experimental Design

**Dataset Construction:** The dataset was synthetically generated to create a controlled environment for testing tokenization dynamics. We generated 10,000 input strings of length  $n$  using a fixed 6-character alphabet  $\Sigma = \{a, b, c, d, e, f\}$ . Each sequence is paired with its reversed counterpart to form a complete reversal equation of the format:

$$\langle \text{input} \rangle = \langle \text{output} \rangle \backslash n$$

yielding a total character length of  $N = 2n + 2$  per instance. Representative examples from the dataset include:

Length  $n = 3$  :    `abc=cba\n`

Length  $n = 2$  :    `bc=cb\n`

Length  $n = 6$  :    `adfcfa=afcfd\n`

For the efficiency experiments (Figure 1), the input sequence length was fixed at  $n = 144$ , yielding  $N = 290$  characters per sequence.

**Model Architecture and Training Setup:** All experiments were conducted using a decoder-only transformer architecture (Vaswani et al., 2017) configured with 4 layers, 4 attention heads, and a hidden dimension  $d_{\text{model}} = 128$ . The vocabulary included 3 functional special tokens: `_` (padding), `=` (delimiter), and `\n` (end-of-sequence). Models were optimized using AdamW with a learning rate of  $10^{-3}$  and a weight decay of 0.1.

Training continued until one of three stopping criteria was met:

1. Achieving  $> 99\%$  validation accuracy (convergence).
2. Reaching early stopping (no validation accuracy improvement for 15 consecutive epochs).
3. Attaining the maximum limit of 100 epochs.

Floating-point operations per epoch (FLOPs/epoch) and cumulative TFLOPs to convergence were directly tracked and calculated during training.

## 4 Results

We evaluated the computational efficiency of the transformer architecture on the character reversal task across 10 independent random seeds. Each seed was evaluated on an independently generated

dataset of 10,000 completion equations using the 6-character alphabet  $\Sigma$ . To ensure a fair comparison, all models were trained until achieving  $> 99\%$  validation accuracy, isolating total compute required to reach convergence.

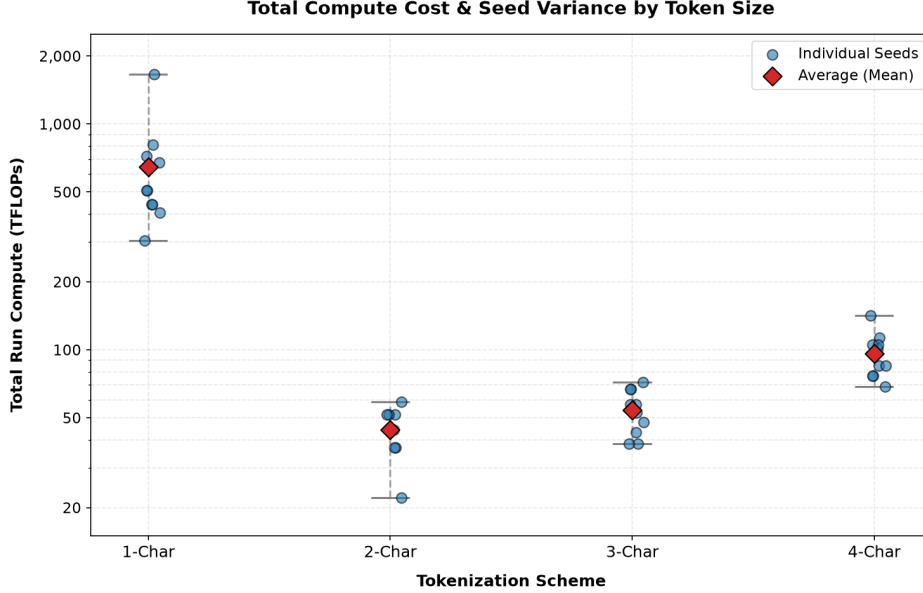


Figure 1: Total computational cost (TFLOPs) to convergence ( $> 99\%$  validation accuracy) across 10 independent random seeds. While the 1-Char scheme suffers from quadratic attention bottlenecks, 2-Char achieves the global optimum at  $\approx 44$  TFLOPs. Coarser schemes (3-Char, 4-Char) incur rising costs driven by parameter expansion.

Table 1: Structural and computational characteristics by tokenization scheme ( $d_{\text{model}} = 128$ ).

| Tokenization | Vocab Size | Embedding Params | Seq. Length | Attn. Ops ( $\mathcal{O}(L^2)$ ) |
|--------------|------------|------------------|-------------|----------------------------------|
| 1-Char       | 6          | 768              | 290         | 84,100                           |
| 2-Char       | 36         | 4,608            | 146         | 21,316                           |
| 3-Char       | 216        | 27,648           | 98          | 9,604                            |
| 4-Char       | 1,296      | 165,888          | 76          | 5,776                            |

As illustrated in Figure 1 and detailed in Table 1, the 1-Char baseline proves severely inefficient, requiring a mean of  $\approx 647$  TFLOPs to converge due to prolonged sequence lengths ( $L = 290$ ). Moving to a 2-Char tokenization scheme yields a dramatic  $14.67\times$  reduction in computational cost, reaching the optimal mean of  $\approx 44$  TFLOPs. However, further coarsening the granularity to 3-Char and 4-Char schemes reverses this trajectory, driving mean compute back up to  $\approx 55$  TFLOPs and  $\approx 95$  TFLOPs, respectively. This U-shaped curve highlights the exact transition point where embedding parameter scaling begins to outweigh the efficiency gains of sequence truncation.

## 5 Analysis

### 5.1 Attention Overhead Reduction (1-Char vs. 2-Char)

The dramatic efficiency gain observed when transitioning from 1-Char to 2-Char tokenization ( $14.67\times$  compute reduction) is primarily driven by the quadratic scaling of self-attention mechanisms,  $\mathcal{O}(L^2 \cdot d_{\text{model}})$ , where  $L$  represents sequence length and  $d_{\text{model}} = 128$ . For an input length of  $n = 144$ , the sequence length decreases from  $L_{1\text{-Char}} = 2(144) + 2 = 290$  to  $L_{2\text{-Char}} = 2(144/2) + 2 = 146$ .

Evaluating the total attention operations across the network yields:

$$\text{Attn Ops}_{1\text{-Char}} = 290^2 \times 128 = 10,764,800$$

$$\text{Attn Ops}_{2\text{-Char}} = 146^2 \times 128 = 2,728,448$$

This truncation yields a  $3.95\times$  direct reduction in attention operations. The remaining compound speedup to reach  $14.67\times$  stems from reduced hidden-state updates across sequence steps and accelerated training convergence due to shortened optimization horizons.

## 5.2 Embedding Parameter Inflation (3-Char vs. 4-Char)

While coarser tokenization continuously shortens sequence length, it introduces exponential vocabulary expansion according to  $|\Sigma|^c$ , where  $c$  is the character  $N$ -gram length and  $|\Sigma| = 6$ . Between 3-Char and 4-Char tokenization, sequence length drops by only  $1.29\times$  ( $L = 98$  to  $L = 76$ ), yielding a modest  $1.66\times$  reduction in attention operations.

Conversely, vocabulary size explodes by  $6\times$  from 216 to 1,296 tokens, causing embedding parameters ( $V \times d_{\text{model}}$ ) to scale from 27,648 to 165,888. This parameter explosion imposes severe memory-bandwidth and gradient-computation penalties, where memory access overhead during forward passes and sparse backpropagation updates across  $6\times$  more parameters outpace sequence truncation benefits—resulting in a net  $1.8\times$  computational slowdown.

## 5.3 Optimality of 2-Char Tokenization

The 2-Char scheme represents the optimal Pareto frontier between sequence length reduction and vocabulary expansion for a 6-character alphabet. It captures the vast majority of quadratic attention savings ( $3.95\times$  reduction) with minimal vocabulary expansion (+30 tokens relative to 1-Char, compared to +180 tokens from 2-Char to 3-Char). Consequently, 2-Char minimizes global compute at  $\approx 44$  TFLOPs, outperforming 3-Char by  $\approx 11$  TFLOPs and demonstrating that optimal tokenization granularity is strictly bounded by alphabet size and model capacity.

# 6 Discussion

The empirical results presented in Figure 1 demonstrate a pronounced non-linear relationship between tokenization granularity and total computational cost to convergence. Rather than monotonic scaling, training compute follows a distinct U-shaped trajectory across the evaluated  $N$ -gram tokenization schemes, with 2-Char emerging as the optimal configuration for minimizing compute overhead.

## 6.1 Attention Bottleneck vs. Embedding Parameter Scaling

The observed U-shaped response curve reflects a fundamental trade-off between two competing computational bottlenecks within the transformer architecture:

1. **Quadratic Attention Bottleneck** ( $\mathcal{O}(L^2 \cdot d_{\text{model}})$ ): At fine tokenization granularities (e.g., 1-Char), character aggregation is minimal, maximizing sequence length ( $L = 290$  for input length  $n = 144$ ). Because self-attention complexity scales quadratically with sequence length, floating-point operations per epoch are overwhelmingly consumed by attention computation (Bulatov et al., 2024). This structural bottleneck drives the high mean computational cost of  $\approx 647$  TFLOPs to reach convergence.
2. **Exponential Embedding Inflation** ( $\mathcal{O}(|\Sigma|^c \cdot d_{\text{model}})$ ): As  $N$ -gram length  $c$  increases, vocabulary size scales exponentially according to  $V = |\Sigma|^c$ . For 4-Char tokenization with  $|\Sigma| = 6$ , vocabulary size inflates to  $V = 1,296$ , causing embedding parameters to expand to 165,888. This shifts the primary computational burden from sequence-level attention to embedding layer management. Furthermore, over-parameterized token spaces suffer from token sparsity, requiring more optimization epochs to saturate representation weights and resulting in an elevated convergence cost of  $\approx 95$  TFLOPs.

The 2-Char tokenization scheme achieves the optimal balance between these competing forces. It captures a substantial  $3.95\times$  reduction in attention operations over 1-Char while incurring a negligible vocabulary expansion (+30 tokens), culminating in the global minimum compute cost of  $\approx 44$  TFLOPs.

## 6.2 Optimization Stability Across Random Seeds

Evaluating variance across 10 independent random seeds provides key insights into the training stability of each tokenization regime:

1. **High Sensitivity (1-Char):** The 1-Char baseline exhibits severe optimization instability, recorded as the highest standard deviation ( $\sigma = 368.02$  TFLOPs). Extended sequence lengths create longer credit assignment paths during backpropagation, making optimization highly sensitive to initial weight initialization.
2. **Optimal Stability (2-Char):** The 2-Char scheme achieves superior optimization stability, yielding the lowest standard deviation ( $\sigma = 9.87$  TFLOPs). Truncated sequence lengths mitigate optimization difficulty while avoiding the sparse gradient dynamics of larger vocabularies.
3. **Re-emerging Instability (4-Char):** As the vocabulary expands to  $V = 1,296$  in 4-Char, variance widens significantly ( $\sigma = 20.74$  TFLOPs)—more than double that of 2-Char and 3-Char. This instability is driven by severe token sparsity, where infrequently updated vocabulary embeddings introduce non-deterministic optimization trajectories.

## 6.3 Limitations and Future Scope

These findings should be interpreted within the scope of our experimental bounds. First, our synthetic benchmark uses a small alphabet size ( $|\Sigma| = 6$ ). In natural language processing or broader algorithmic settings where  $|\Sigma| \approx 100$ , fixed  $N$ -gram tokenization becomes intractable ( $V = 10^6$  for 3-Char and  $V = 10^8$  for 4-Char), quickly exceeding memory constraints. Second, our dataset features uniform sequence structures and character distributions. On non-uniform or unstructured corpora, adaptive algorithms such as Byte-Pair Encoding (BPE) remain essential for dynamically balancing vocabulary capacity against sequence truncation.

## 7 Conclusion

In this work, we empirically evaluated the computational efficiency of character  $N$ -gram tokenization schemes on a controlled string reversal task ( $n = 144$ ,  $|\Sigma| = 6$ ). Our experiments demonstrate that tokenization choice is a dominant factor in training efficiency: the optimal 2-Char configuration requires only  $\approx 44$  TFLOPs to converge, whereas a naive 1-Char baseline incurs a  $14.67\times$  computational penalty due to quadratic self-attention costs. Conversely, coarser schemes (3-Char and 4-Char) suffer from exponential vocabulary expansion and parameter inflation, establishing a clear U-shaped relationship between token granularity and total compute.

Future work will expand this framework by evaluating varying sequence lengths, larger alphabet sizes, and non-uniform data distributions. Furthermore, we aim to formulate an analytical cost estimator to predict the optimal tokenization scheme prior to training. Finally, comparing fixed  $N$ -gram schemes against adaptive subword algorithms (e.g., BPE) across unstructured datasets will help establish rigorous guidelines for tokenization selection in non-natural sequence modeling.

## References

- Bulatov, A., Kuratov, Y., Kapushev, Y., & Burtsev, M. (2024). Beyond attention: Breaking the limits of transformer context length with recurrent memory. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(16), 17700–17708. <https://doi.org/10.1609/aaai.v38i16.29722>
- Dotan, E., Jaschek, G., Pupko, T., & Belinkov, Y. (2024). Effect of tokenization on transformers for biological sequences. *Bioinformatics*, 40(4), btae196. <https://doi.org/10.1093/bioinformatics/btae196>
- Sennrich, R., Haddow, B., & Birch, A. (2016). Neural machine translation of rare words with subword units. *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 1715–1725. <https://doi.org/10.48550/arXiv.1508.07909>

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 5998–6008.

### **Statement on Artificial Intelligence Usage**

In accordance with research transparency standards, Large Language Models (LLMs)—specifically Claude and Gemini—were utilized as assistive tools during this project. Specifically, LLMs assisted in implementing data synthesis and training scripts, framing initial hypotheses regarding parameter scaling between 3-Char and 4-Char schemes, and refining the structural outline and stylistic editing of the manuscript draft. The core experimental design, empirical data collection, statistical validation, and final interpretation of results remain the original work of the author.